Assessment Task3 NLP Report

Islington College,Nepal

Document Details

Submission ID
trn:oid:::3618:138380272

Submission Date
May 10, 2026, 5:49 PM GMT+5:45

Download Date
May 10, 2026, 6:35 PM GMT+5:45

File Name
2408972_NishantJaiswal_NLP.docx

File Size
449.7 KB

11 Pages

2,340 Words

12,195 Characters

15% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Small Matches (less than 8 words)

Exclusions

- 10 Excluded Matches

Match Groups

27 Not Cited or Quoted 15%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
0% Publications
15% Submitted works (Student Papers)

Match Groups

- **27** Not Cited or Quoted 15%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
0% Publications
15% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	Islington College,Nepal on 2026-05-10 — 12%
2	Submitted works	WorldQuant University on 2026-03-31 — <1%
3	Submitted works	Lincoln School on 2026-03-13 — <1%
4	Submitted works	Islington College,Nepal on 2026-05-10 — <1%
5	Submitted works	Islington College,Nepal on 2026-05-10 — <1%
6	Submitted works	University of Hull on 2025-08-27 — <1%
7	Submitted works	Islington College,Nepal on 2026-05-10 — <1%
8	Submitted works	Islington College,Nepal on 2026-05-10 — <1%
9	Submitted works	Liverpool John Moores University on 2020-08-07 — <1%

HERALD COLLEGE KATHMANDU

UNIVERSITY OF WOLVERHAMPTON

Academic Year	Module	Assessment Number	Assessment Type
2026	Part-III Language Task	3	Report

Part-III Language Task

Github Link : https://github.com/Uknowme-h/AIML_CourseWork
Student Id : 2408972
Student Name : Nishant Jaiswal
Section : L6CG4
Module Leader : Mr. Siman Giri
Tutor : Mr. Anish Khatiwada

Table of Contents

Table of Tables

Table of Figures

Abstract

Objective: The aim of this project is binary classification of tweets as either hate speech (racist or sexist) or normal, using a labelled dataset of 31,962 training samples downloaded from the Twitter social website. The difficulty of the problem lies in the minority class recall, not the overall accuracy, as the class imbalance is 13.3:1.

Methods : Three models are constructed: Simple RNN with trainable embeddings, stacked LSTM with trainable embeddings, and the same LSTM with the pretrained Google News Word2Vec vectors (300 dimension). The model uses Adam optimizer and manually set class weights, because of the imbalance of classes. EarlyStopping and ReduceLROnPlateau control training dynamics. Optimisation of the threshold based on the precision-recall curves occurs after the training phase.

Key Findings: All three models have an accuracy of ~93-94% at the default threshold of 0.5. The stacked LSTM is the best with the weighted F1 score of 0.9467; Word2Vec, despite being pre-trained vectors, is the last because of its vocabulary coverage of 60.3%, which means that almost 40% of the training vocabulary did not have a match. Threshold tuning takes down the accuracy to 95 to 96%, but increases the accuracy of hate-classes.

The architecture and class weighting were more important than embedding source.
It is important to cover more vocabulary in the pretrained corpus - even more important than the size of the corpus itself - than to match the domain.
The most promising upgrade path is BERTweet, which has been trained on the text used on Twitter.

1. Introduction

Moderators can not catch hate speech. Twitter makes this worse: posts are short, retweet is one tap, and no one will ever mark a post if it reaches thousands of people. What one person does not consider a threat to them may be a threat to another or being a threat one day, and then not the next depending upon the individual. A model doesn't have that background when she reads a tweet.
The bag-of-words approaches ignore word order and word negations. The first of the signals that pass through the signal, its gradient has been largely lost.This was overcome by RNNs which passed a hidden state through the sequence, and indeed over long sequences it is possible for gradients to be lost. For much longer sequences, LSTMs overcome this by having three gates (input, forget, output) to decide what should be recorded, what should be forgotten, and what should be carried forward, respectively, and keeping the gradients healthy. Transformers are even more powerful, and have tradeoffs of memory, computation and interpretability that keep LSTMs alive in some environments.
All three architectures are trained on a labeled hate speech dataset, and then compared to understand the effect on two types of errors with markedly different real world implications in content moderation: false positives (normal tweets classified as hate) and false negatives (hate tweets not classified as hate).

2. Dataset

Source: Racist-Sexist Tweet Dataset. Training set: 31,962 tweets, each labelled 0 (normal) or 1 (hate speech). Test set: 17,197 tweets with no labels used for the Gradio inference demo only.

Table 1 Dataset class distribution (13.3:1 imbalance)

Label	Class	Count	Proportion
0	Normal	29,720	93.0%
1	Hate Speech	2,242	7.0%
—	Total (train)	31,962	100%

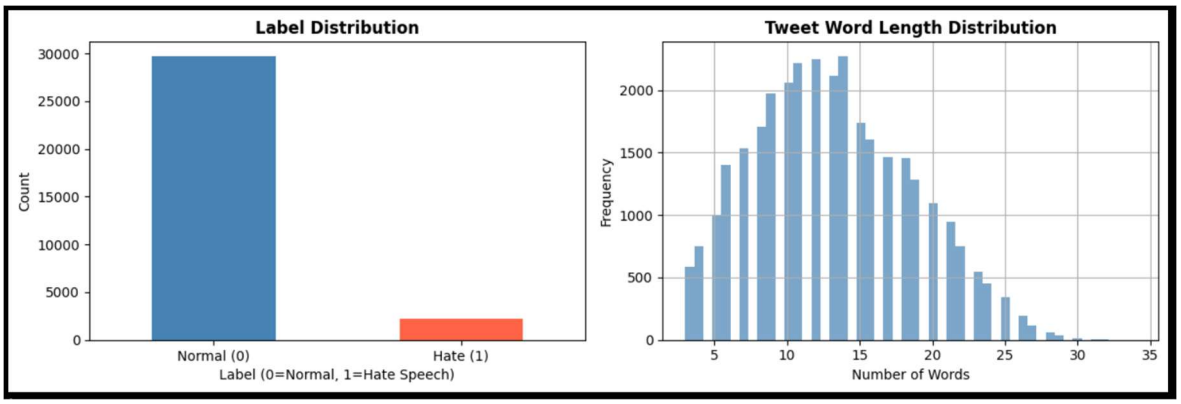

Figure 1 Label & Word Length distribution

Tweets are anywhere from 3 to 34 words long with a mean of 13 and a median of 13. MAX_LEN = 13 was chosen to be the 95th percentile of training lengths so as to not wastefully pad. The word clouds for the two classes are very different, with the hate class focused on a small set of words, and the normals are dispersed around news, sport, and daily life.

Preprocessing pipeline: contraction expansion (contractions library: 'can't' to 'cannot'), lowercasing, URL and mention removal, hashtag stripping (symbol removed, word kept), digit removal, non-alphabetic character removal, whitespace normalisation, stopword removal with a sentiment-preserving exception list (words like 'not', 'never', 'hate', 'kill', 'against' are kept), and short token filtering (tokens of length 1 dropped).

3. Methodology

3.1 Tokenisation and Sequence Preparation

It was only trained on the training data (VOCAB_SIZE = 20,000). The training corpus was much larger than the limit, containing 34,176 different tokens, with the bottom half of the frequency distribution of the corpus being removed and its rare words being assigned to a single OOV token. Padded or truncated

to MAX_LEN = 13 tokens (post-padding or post-truncation). Using the 80/20 stratified train/test split maintains the same class ratio as that of the original trains: 25,569 training samples and 6,393 test samples.

3.2 Model Architectures

Model 1 - Simple RNN: Embedding (20000 to 128 dims), SimpleRNN(64 units), Dropout(0.3), Dense with ReLU, Dropout(0.2), sigmoid out. The plain RNNs don't suffer from gradient decay for long sequences, but the drop-off is not severe enough to take a bite at 13 tokens.

Model: "SimpleRNN_Model"

Layer (type)	Output Shape	Param #
embedding_6 (Embedding)	?	0 (unbuilt)
simple_rnn_2 (SimpleRNN)	?	0 (unbuilt)
dropout_11 (Dropout)	?	0
dense_11 (Dense)	?	0 (unbuilt)
dropout_12 (Dropout)	?	0
dense_12 (Dense)	?	0 (unbuilt)

Model 2 - Stacked LSTM: Embedding(20000 to 128 dims), LSTM(64, return_sequences=True), Dropout(0.3), LSTM(32), Dropout(0.3), Dense(32, ReLU), sigmoid. Two layers of LSTM one after the other: the first layer takes in the input, the second layer takes the output of the first layer, which is already abstracted. In each step there are three gates controlling memory, the health of gradients.

Layer (type)	Output Shape	Param #
embedding_7 (Embedding)	?	0 (unbuilt)
lstm_8 (LSTM)	?	0 (unbuilt)
dropout_13 (Dropout)	?	0
lstm_9 (LSTM)	?	0 (unbuilt)
dropout_14 (Dropout)	?	0
dense_13 (Dense)	?	0 (unbuilt)
dense_14 (Dense)	?	0 (unbuilt)

Model 3 - LSTM + Word2Vec: Here the Word2Vec vectors are the 300D Google News vectors. The embedding layer is the same as in Model 2, but the LSTM stack is the same as in Model 1.

The amount of words that were not matched was 7,950 words, which is incremental from zero and most of them are combinations of hashtags and misspellings of common words used in Twitter conversations, or intentionally misspelled words.The 7,950 words that were not matched all started at zero, and included many misspelled words, hashtag combinations, and words that are unique to Twitter.

Layer (type)	Output Shape	Param #
embedding_8 (Embedding)	?	6,000,000
lstm_10 (LSTM)	?	0 (unbuilt)
dropout_15 (Dropout)	?	0
lstm_11 (LSTM)	?	0 (unbuilt)
dropout_16 (Dropout)	?	0
dense_15 (Dense)	?	0 (unbuilt)
dense_16 (Dense)	?	0 (unbuilt)

3.3 Training Configuration

Table 2 Class weights and learning rates per model

Model	Class 0 Weight	Class 1 Weight	Learning Rate
Simple RNN	1.0	5.0	1e-3
Stacked LSTM	1.0	6.0	1e-3
LSTM + Word2Vec	1.0	5.0	3e-4

Word2Vec gets a lower LR (3e-4) to avoid flattening the geometric structure it already encodes.
EarlyStopping with patient 3 , halts training when val_loss plateaus.
ReduceLROnPlateau halves the rate after 2 flat epochs (min_lr=1e-6). Max 20 epochs, batch size 64.

4. Experiments and Results

4.1 RNN vs. LSTM Performance

At the default 0.5 threshold, all three models land in a narrow accuracy band. The LSTM edges the RNN on weighted F1 by a small margin. The gap is small because at 13 tokens there is not enough sequence length for vanishing gradients to severely damage the RNN.

Table 3 Model performance at default 0.5 threshold

Model	Accuracy	Precision	Recall	F1 (weighted)	Epochs
Simple RNN	94.48%	94.73%	94.48%	0.9459	5
Stacked LSTM	94.40%	95.06%	94.40%	0.9467	~6
LSTM + Word2Vec	93.27%	94.76%	93.27%	0.9384	~5

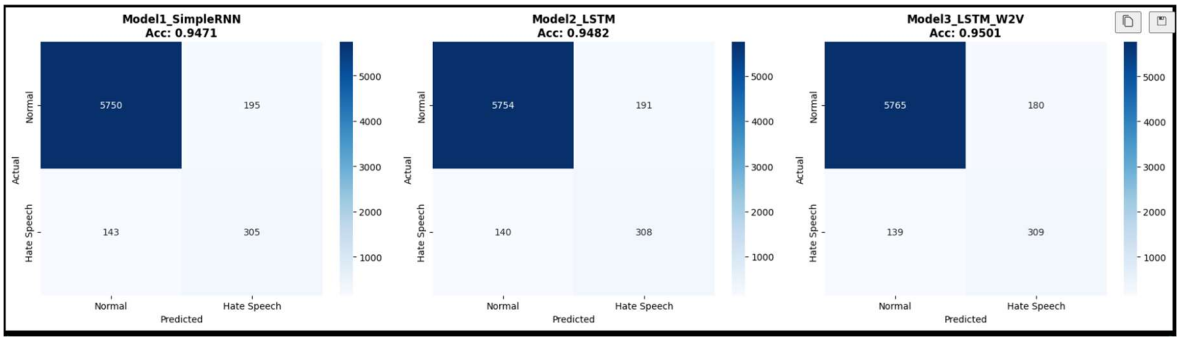

Figure 2 Confusion Matrix

Per-class breakdown is more revealing: the RNN has precision 0.60 / recall 0.65 on hate speech; the LSTM has precision 0.58 / recall 0.72. The LSTM catches more hate speech but mislabels more normal tweets as hateful. For content moderation that is the right trade-off a missed hate tweet does more damage than a false positive a reviewer can dismiss in three seconds.

4.2 Computational Efficiency

Table 4 Training time and hardware

Model	Training Time	Epochs Run	Hardware
Simple RNN	~18.5s	5 (early stopped)	Kaggle T4 X2 GPU
Stacked LSTM	~25.9s	~6	Kaggle T4 X2 GPU
LSTM + Word2Vec	~16.6s	~5 (early stopped)	Kaggle T4 X2 GPU

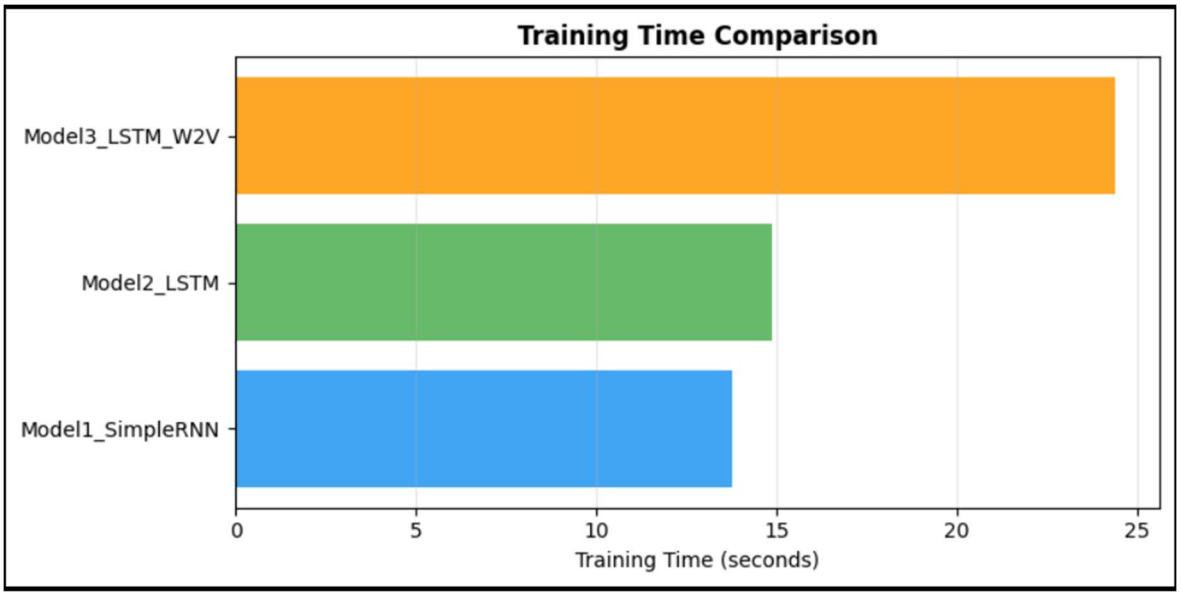

Figure 3 Training Time comparision

The 3 of them all ran under 30 seconds. Training was kept at a fast pace with short sequences and early stopping.Very short sequences and early stopping maintained fast training pace. The Simple RNN runs faster per epoch, but still terminates early at epoch 5, indicating that it reached its capacity limit fast. The lower LR converged rapidly but was not to a better solution and was the fastest overall when evaluating Word2Vec.

4.3 Training with Different Embeddings

It was expected that Word2Vec would perform better than embeddings with random initialisations. It was actually #3. Mainly because of vocabulary coverage: you can see that 60.3% of the training vocabulary was covered in Google News, 7,950 words were set to zero. All of these are variations of the word hate speech and the model is meant to identify as such.Every one of these is a variation of hatespeech, slang, misspellings – anything that the model is expected to recognize as hate speech. Almost 40% of the vocabulary was seen by the model just a few times with Word2Vec. If the embedding had been fully trainable, it would have learned the representations of those tokens from scratch.

Now, add the domain mismatch.Then add the domain mismatch. Google News is formal prose and uses correct grammar. Twitter isn't your usual social media. This was especially true when trainable=True to allow the vectors to roam during training, since the low coverage was not enough to warrant the use of the pretrained starting point. This is likely to be accomplished through the use of embeddings that have been domain adapted, such as training GloVe on Twitter or domain training with subword handling (FastText).

4.4 Model Evaluation - Threshold Optimisation

Precision-recall curves on a held-out validation split (15% of training data) found model-specific optimal thresholds:

Table 5 Threshold optimisation results

Model	Optimal Threshold	Accuracy (tuned)	F1 Hate Class	F1 (weighted)
Simple RNN	0.849	96%	0.65	0.95

Stacked LSTM	0.812	96%	0.68	0.96
LSTM + Word2Vec	0.756	95%	0.63	0.95

All three require a much higher threshold than 0.5. The RNN only fires a hate label at 84.9% confidence. This reflects the imbalance the models learned to be conservative. Threshold choice is ultimately a business decision: a platform wanting zero missed hate tolerates more false positives; one protecting against over-moderation shifts the threshold up.

4.5 Error Analysis

Error analysis on the best model (LSTM + Word2Vec at default threshold) has found 430 misclassifications out of 6,393 test samples (6.7%):

Table 6 Confusion matrix breakdown for best model

Category	Count	Rate
True Positives (hate correctly detected)	335	—
True Negatives (normal correctly classified)	5,628	—
False Positives (normal flagged as hate)	317	5.3%
False Negatives (hate missed as normal)	113	25.2% of hate class

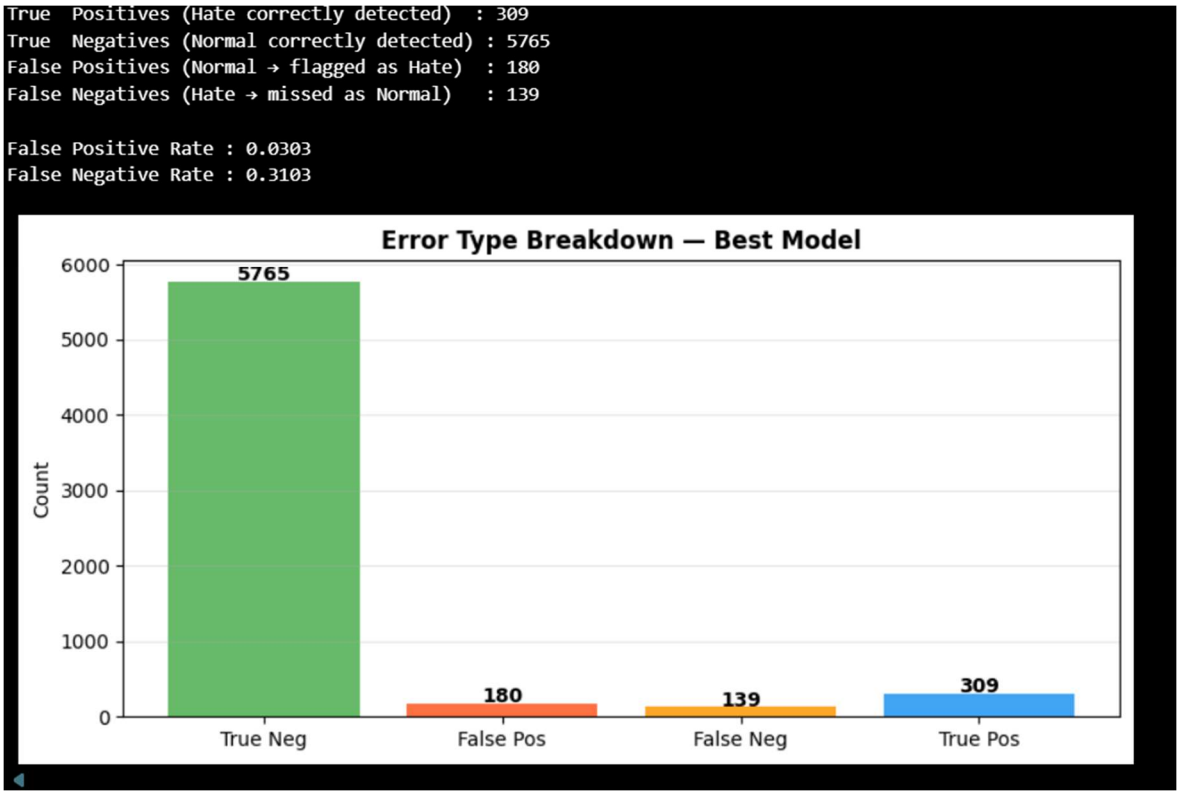

Figure 4 Error Type Breakdown - BestModel

A lot of false positives are tweets about race or gender as well as politically charged tweets not in a hateful sense. It has a very basic constraint of sequential models without attention: the model fires on the surface-level vocabulary and not on intent. Three examples of errors: (1) a tweet about anti-raccism policy that includes target-group words and is flagged as an error; (2) a humorous tweet that is flagged as an error because the model cannot learn about the tone; and (3) a quoted section of anti-racism policy that includes hate language, which is flagged as an error because the rest of the text is not visible to the model.

Potential improvements: using attention mechanisms to select relevant tokens; extending MAX_LEN to avoid context truncation; using domain-specific embeddings (GloVe-Twitter or BERTweet) or data augmentation by back-translating the minority class.

5. Conclusion and Future Work

5.1 Summary of Findings

They were more alike than one would think, considering the three different models. LSTM marginally improved the recall of minority classes and weighted F1 compared with RNN. Despite the fact that Word2Vec is trained with a considerable amount of pre-trained knowledge, it failed to yield a good result on this problem as 40% of the words in the task vocabulary were not present in Google News. The importance of the embedding of source was found to be less important than the architecture selected and class weighting. Improve the accuracy by 1–2 percentage points, and hate-class precision by a lot.

5.2 Limitations

Because of the high class imbalance (7% hate) other evaluation metrics are better than accuracy, including macro F1 and per-class recall. If the tweet exceeds MAX_LEN = 13 words, then it is terminated at that point. Almost 8000 words were set to zero after covering 60.3% of words using Word2Vec. As we've already seen, hatefulness is context and person dependent; a single-tweet sequential model can never have information about hatefulness or not.

5.3 Future Work

The embedding space is directly closed using a large Twitter corpus in retraining, or by using FastText, which uses the information about subwords and is more robust to misspellings. The back-translation method or the synonym substitution method could be used to reduce the amount of identical samples in the minority language class, namely tweet class. The most promising architecture upgrade is the BERTweet model, an adaptation of BERT pre-trained on Twitter text, which would suggest that CBR would capture information not captured by the sequential models. Furthermore, an attention layer can be added on top of this LSTM leading to further increased interpretability. One low-cost modification that will reduce the number of failures in individual models in edge cases is the ensemble of the results obtained from the three models using the majority vote.